PAYMENTS ENGINEERING · ARCHITECTURE PLAN

Payout Simplification — Platform-Held Settlement for 2027+ Bookings

How card, card-reader, and payment-link charges for 2027+ tour dates settle entirely on the platform's own Stripe balance instead of an automatic destination-charge split — while the guest still sees the tour operator's own name on their statement, in every country the platform operates in.

Platform: NAT Tour Backend — Laravel 11, Stancl/Tenancy v3, Stripe Connect

Prepared for: Devat Karetha

Prepared: September 19, 2026

Revised: October 6, 2026 — Section 8.5 correction

Branch: `feature-stripe-2027-payout-simplification`

Status: Foundation, Phase A/B/C, and the statement-descriptor fix (corrected and extended to every payment route on October 6, 2026 — see Section 8.5) are shipped and tested; Auto Payout integration and full checkout-redirect verification remain open — see Section 15

CONFIDENTIAL — INTERNAL ENGINEERING PLANNING DOCUMENT

Revision History

A running change log of every phase and revision of this project, in chronological order, cross-referenced to the real commits on this branch. See Section 7 for the structured component-by-component status and Section 8 for the statement-descriptor investigation in full narrative detail.

Foundation laid (`5648f3c7f`, `741e319da`, `ef9c242db`, `cb45a238c`). `PaymentModeResolver` introduced as the single place the 2027-01-01 boundary is expressed in code, gated by a per-tenant kill-switch (`PLATFORM_HELD_PAYOUT_ENABLED`, reusing the existing `hasSetting()` mechanism already used for `AUTO_PAYOUT` / `DISPUTE_REVERSAL`) and failing closed to `DESTINATION` whenever that setting is missing or off. `PaymentModeInference::fromStripeObject()` added alongside it for the handful of call sites that only have a raw Stripe object, not a local `Transaction` row, in scope. `buildConnectedPaymentData()` / `buildRefundParams()` / `refundPayment()` gained a `$mode` parameter (required, no default, on the refund side specifically — a forgotten default there would silently send `reverse_transfer` against a charge with nothing to reverse). `transactions.payment_mode` column added (enum, default `'destination'`, correctly backfilling every historical row). `TransactionService`'s payment- and refund-writing methods and `CancelTransactionUpdateService::run()` threaded with the mode from each caller's already-in-scope `Transaction` row — no new queries. This is the "persist mode once, never re-derive" principle that governs every fix in this project: a booking's settlement mode is decided once, at charge-creation time, and every later refund/cancellation/dispute reads that decision back off the row rather than re-resolving it against the booking's current tour date.

Two live production refund-calculation bugs found and fixed the same week, independent of Payout Simplification's own new code but surfaced while auditing the same refund paths (`559d4c825`, `36f709684`). A guest cancellation on tenant `kens` produced "Refund amount ($180.49) is greater than unrefunded amount on charge ($102.92)" — traced to `TicketRefundCancelService.php` computing its net refund as `$transTicketCost + $oldRefundKeepUs` instead of netting out the prior partial refund (`$transTicketCost - $ref`); an `$oldRefundKeepUs` figure that was never actually refunded (a pure reporting artifact from the "Full Refund minus Fees" pax-removal path in `TourBookingRepository.php`) was being added back into a real Stripe refund amount. A structurally identical bug in `WeatherRefundCancelService.php` was caught immediately after by re-deriving the same booking's numbers by hand and querying why a follow-up refund had paid $94.47 instead of the correct $86.02 — fixed by netting out both the prior refund *and* the previously-kept fee (`$alreadyKeptFee`, accumulated from the booking's own `RefundHistory` rows) from `$transFullAmount`. Both are cash-correctness bugs in refund arithmetic that predate this project; they were found only because this project required reading every refund call site closely enough to notice the amounts didn't reconcile.

Phase C — Payment Link (`a71ba1dff`). `PaymentLinkService::createPaymentLink()` rebuilds its Stripe Checkout Session fresh on every visit, not once at link-creation time, so mode is resolved inside the method itself at each rebuild via `PaymentModeResolver::resolve()` across the link's own booking dates, and threaded into both `buildConnectedPaymentData()` and the pre-existing `stripOnBehalfOfForCountry($params, $mode)` helper (a 4-month-old Canada hotfix, unrelated in origin to this project — see Section 8). `tests/Unit/Payments/PaymentLinkPlatformHeldModeTest.php` added.

Unrelated JS bug fixed in passing (`70e0cfd65`). A guest-reported `ReferenceError: combinedAmount is not defined` in the add-ons bulk-payment modal's card-reader flow (`tour-booking-addons.js`) — a variable scoped to the wrong method. Fixed by re-deriving it locally, mirroring the sibling method's already-correct pattern. No connection to settlement mode; fixed because it blocked a user-reported live flow during the same work session.

payment_mode sweep on refund/cancellation writes, plus an 8-row live data correction on `kens` (`0252d5b3e`). A user-reported screenshot showed a cancelled add-ons booking's refund rows stored with `payment_mode = 'destination'` despite having actually settled correctly — the real bug was that `CancelTransactionUpdateService::run()`, the single shared method behind all four cancellation-refund services (Ticket Refund, 100% Refund, Partial Refund, Don't Refund), never set `payment_mode` at all, silently inheriting the DB column's `'destination'` default regardless of what the original charge actually was. Fixed by making `$mode` a required parameter across all 29 call sites project-wide (`WeatherRefundCancelService.php` ×8, `TicketRefundCancelService.php` ×6, `PartialRefundCancelService.php` ×8, `DontRefundCancelService.php` ×6, plus one more), each supplying the real transaction's own persisted `payment_mode`. Cross-referenced against sibling `status='Completed'` rows sharing the same `transaction_number`, 8 already-wrong rows were found and corrected directly on `kens` (transaction 309201 for booking 348618, plus 7 more: 309174, 309182, 309168, 309176, 309185, 309171, 309179) — all had been written as `'destination'` when the originating charge was actually `'platform_held'`. New test: `tests/Unit/Bookings/CancelTransactionUpdateServiceFeeAmountTest.php`'s sibling coverage confirmed the mode threading; a dedicated payment_mode-specific regression accompanies the same commit.

Phase B — Card Reader (`31cbc5498`). Both Terminal flows wired: the web/admin path (`CardReaderRepository::getPaymentIntent()`) via a new `resolveModeFromInput()` that reads `metadata.dates` (cart) or resolves a single tour date from `metadata.booking_id`, falling back to `DESTINATION` when neither is present (an already-open browser tab mid-rollout has no way to supply the new field, and the resolver's own fail-closed contract already makes "no date available" mean "cannot be platform-held"); and the mobile-app Terminal reader

(`TourBookingRepository::stripeCreatePaymentIntent()` , V1 and V3, both cart and single-booking branches). A pre-existing, independent bug was found and fixed in the same pass: `CardReaderRepository.php` hardcoded `'application_fee_amount' => $input['fees'] ?? 0` into the base payload *before* `buildConnectedPaymentData()` 's return value was spread in — meaning even with mode support added, this stray key would have survived untouched into a platform-held charge. Deleted outright rather than patched with a post-hoc `unset()` . JS payloads (`backend-card-reader.js` , `tour-booking-pos.js`) updated to send the new `meta.dates` field. New test: `tests/Unit/Payments/CardReaderPlatformHeldModeTest.php` (4 cases covering both metadata shapes and the fallback).

The Canada statement-descriptor bug — found, and an earlier verification pass corrected (`422dc3ddf`). See Section 8 for the full narrative. In short: an earlier verification pass had concluded `on_behalf_of` with no `transfer_data` works unconditionally, regardless of the connected account's `card_payments` capability — every prior test had either never confirmed a payment method or stalled at 3DS `requires_action` before Stripe would ever have run the actual check. A real Stripe Dashboard screenshot of a stuck test transaction with a blank statement descriptor prompted re-verification using `off_session: true` to force a real charge to completion, which surfaced Stripe's actual rejection: `on_behalf_of` alone still requires `card_payments` , even with no `transfer_data` present. `buildConnectedPaymentData()` 's `PLATFORM_HELD` branch rewritten to gate `on_behalf_of` on the same `connectedAccountHasCardPayments()` check the destination-mode branch already used, falling back to a full `statement_descriptor` override (a property of the platform's own charging account, needing no connected-account capability at all) for accounts that lack it — which today means most Canadian operators on this platform. Verified end-to-end with real completed charges against both a US tenant (`dixies` , has the capability) and the CA tenant (`aswta` , does not). `tests/Unit/Payments/StripePaymentGatewayModeTest.php` rewritten to assert against real tenant Stripe destination IDs instead of a fake account string.

A second, independent near-miss found while writing this document (2026-09-19, no separate commit — corrected via migration `2026_09_18_000003_fix_stale_platform_held_payout_setting.php`). The tenant `dixies` was found to already carry a `PLATFORM_HELD_PAYOUT_ENABLED` settings row with `value=1` — left over from an unrelated, already-reverted "Payout Simplification proof-of-concept" built on a prior branch that happened to reuse the exact same settings key. Because the Foundation migration only inserts its safe `value=0` default when the key does not already exist, this stale row was silently preserved instead of overwritten, meaning the kill-switch had been reading as **enabled** on `dixies` for the entire duration of this project so far. No real charge was misrouted — confirmed by `SELECT COUNT(*) FROM transactions WHERE payment_mode = 'platform_held'` returning zero on every tenant — because the actual PaymentIntent-creation call sites only started consulting `PaymentModeResolver` partway through this same work session, and no 2027-dated booking has been sold yet on any tenant. A targeted fix-up migration resets any row whose comment doesn't match this feature's own canonical text back to `value=0` , leaving alone any row that already carries this feature's own comment (i.e. a deliberate later toggle). This is the reason Phase A's lifecycle proof and Phase B/C's rollout plan (Section 9) both call for confirming the kill-switch's live value on every target tenant before enabling anything, rather than trusting the migration's default in isolation.

fee_amount / application_fee_id null-vs-zero correction, scoped to fee-preserving refund types only (`65e5e47f4`). The request was to ensure `fee_amount` / `application_fee_id` are stored as `null` , never `0` or an empty string, whenever a refund doesn't actually refund the platform's fee — explicitly **excluding** 100%/Full Refund, which genuinely does refund the fee and must keep its real value. Fixed at the shared-method level rather than per call site, given the caller count: `TransactionService::storeRefundInTransaction()` , `storeAddonsRefundInTransaction()` , and `storeCustomChargeRefundInTransaction()` each now compute a local "store as null unless genuinely non-zero" value before writing; `CancelTransactionUpdateService::run()` got the same normalization directly. 6 new tests added to `tests/Unit/Bookings/TransactionServiceTest.php` (zero-fee-becomes-null and real-fee-is-preserved, for all three methods) and 3 to `CancelTransactionUpdateServiceFeeAmountTest.php` .

Auto Payout exclusion confirmed already correct, mechanism explained (no code change this pass). Following a question about why `payment_mode = 'destination'` filters exist on three Auto Payout query sites (`PayoutApprovalService.php:109` , `PayoutAggregateService.php:189` , `AutoPayoutController.php:637`), confirmed and documented (Section 11) that these guard against a real future-correctness gap: Auto Payout sums local `transactions` rows with no inherent awareness of settlement mode, and its pre-send balance check is a live Stripe balance query on the connected account — a blunt "don't overdraw" safety net, not a source-of-truth check — that would not by itself catch a platform-held transaction being included in a payout computation once such rows exist.

This document. Written to formally record the architecture, the locked-in scope decisions, the statement-descriptor investigation (a genuine correction of an earlier finding, worth recording plainly rather than quietly superseding), the live incidents fixed along the way, and the items still open before this can be considered fully shipped (Section 15).

Correction — statement-descriptor coverage (October 6, 2026; uncommitted at time of writing). A later review found that the Canada fix described in Section 8 reached Payment Links only: the rewritten `buildConnectedPaymentData()` branch produces a full `statement_descriptor` only when the caller passes the operator's descriptor in, and 18 of the 19 call sites never did. On a Canadian

account that left the guest statement as the platform's name plus a suffix (`TOURS* ASWTA`). Fixed once, at `StripePaymentGateway::createPayment()`, and verified in Stripe test mode. Full narrative in Section 8.5.

Contents

1. Executive Summary
2. Current Architecture — Destination Charges
3. Scope & Locked-In Decisions
4. Target Architecture — Platform-Held Settlement
5. The 2027 Gate — `PaymentModeResolver` & Kill Switch
6. New & Changed Components — Foundation
7. Phase-by-Phase Wiring (Card / Card Reader / Payment Link)
8. The Canada Statement-Descriptor Investigation
9. Live Production Incidents Found Along the Way
10. The fee_amount / application_fee_id Correction
11. Auto Payout Exclusion
12. Data Model Changes
13. Testing Summary
14. Risk Register
15. Open Items — What's Pending
16. Appendix A — Key File & Call-Site Reference
17. Appendix B — Stripe Connect Glossary

1. Executive Summary

Today, every card, card-reader, and payment-link charge on the platform is created as a Stripe **destination charge**: the PaymentIntent carries `transfer_data.destination` and `application_fee_amount`, and Stripe automatically splits and pays the tour operator's share into their connected Stripe account the instant the charge succeeds. **Payout Simplification** changes this for any booking whose **tour date** falls on or after **2027-01-01**: the full charge instead lands on the platform's own Stripe balance, with no automatic split at all. The operator is paid later, on whatever schedule their tenant already runs, through the existing Auto Payout system — unchanged in mechanism, simply now also aware that a platform-held transaction never landed money in the operator's account in the first place.

This is deliberately a smaller change than it might sound. A related, more ambitious idea — an instant per-booking Stripe `Transfer` to the operator, fired synchronously at charge time — was explored on a sibling branch (`docs/payments/stripe-2027-transfer-plan.html`) and shelved in favor of this project, because Payout Simplification needs none of that new machinery: no per-booking `Transfer`, no per-booking transfer-reversal-on-refund logic, no `transfers` ledger table. The only thing that changes at charge time is *which Stripe parameters are sent* — everything downstream (refunds, cancellations, disputes, and ultimately how the operator gets paid) continues to work off the same `transactions` table it always has, just reading a new `payment_mode` column to know which shape the original charge actually was.

SCOPE CONFIRMATION

This round covers **card, card-reader (Stripe Terminal), and payment-link charges only**, gated purely by tour date. Guest tips are explicitly excluded from this round regardless of tour date (Section 3). Building the "pay the operator for a platform-held booking" leg of Auto Payout is also explicitly a future round — this round's job is to make sure Auto Payout does not *mis*-pay in the meantime (Section 11), not to build the new payout mechanism itself.

CONFIRMED — CLEAN FORWARD CUTOVER, GATED BY A KILL SWITCH

No 2027-dated booking has been sold on any tenant yet, so there is no historical platform-held transaction to reconcile or backfill — every booking today, and every booking created before this ships fleet-wide, keeps running on the existing destination-charge model unchanged, forever. The feature is additionally gated by a per-tenant `PLATFORM_HELD_PAYOUT_ENABLED` kill switch that fails closed (Section 5), so the 2027-01-01 boundary itself is not the real rollout gate — enabling the setting, tenant by tenant, is. A real near-miss with this exact setting (Section 9) is why that distinction matters in practice, not just in theory.

THE FINDING THAT MATTERED MOST THIS ROUND

An earlier verification pass concluded the guest-facing statement descriptor (the tour operator's own name, never the platform's) would work correctly in every country under the new model, including Canada, without any extra logic. That conclusion was wrong, and was only caught because a real Stripe Dashboard screenshot showed a blank descriptor on a stalled test transaction. Section 8 documents the mistake, the correct mechanism, and the fix — in full, not glossed over — because it is the clearest example in this project of why "it worked in a quick test" and "it was verified against a real completed charge" are not the same claim.

2. Current Architecture — Destination Charges

All card-based money movement funnels through `App\Contracts\Payments\PaymentGatewayInterface`, implemented by `App\Services\Payments\StripePaymentGateway`, resolved via `app(PaymentGatewayInterface::class)` throughout repositories and services. The connected-account parameters for any given charge are built by one centralized method, `buildConnectedPaymentData(string $destinationId, int $feeAmountCents, ?string $statementDescriptor, string $mode)`, which every PaymentIntent/Checkout-Session-creation call site is expected to call rather than assembling `transfer_data` / `application_fee_amount` / `on_behalf_of` by hand.

For today's destination-charge mode (still the only mode for pre-2027 tour dates, and for every booking until a tenant's kill switch is turned on), that method returns:

```
$data = [
    'application_fee_amount' => $feeAmountCents,
    'transfer_data'          => ['destination' => $destinationId],
];
// on_behalf_of requires card_payments capability; omitted for accounts that only have transfers (e.g. Canada)
if ($this->connectedAccountHasCardPayments($destinationId)) {
    $data['on_behalf_of'] = $destinationId;
}
if ($statementDescriptor !== null) {
    $data['statement_descriptor_suffix'] = $statementDescriptor;
}
```

Refunds against a destination charge send `reverse_transfer` / `refund_application_fee` so Stripe pulls back the corresponding slice of the automatic transfer and fee at the same time. Disputes lost against a destination charge automatically reverse the transfer as part of Stripe's own dispute-loss handling. None of this changes for any pre-2027 booking, on any tenant, regardless of whether that tenant's kill switch is on — the mode is decided per booking, at charge-creation time, not per tenant globally.

3. Scope & Locked-In Decisions

3.1 Mixed-date carts follow the later date

A single cart/PaymentIntent can bundle multiple bookings, each with its own tour date. `PaymentModeResolver::resolve(iterable $tourDates)` is conservative: if *any* date in the cart is 2027-01-01 or later, the whole charge settles as platform-held. Stripe cannot split one charge's settlement two ways, and treating an earlier-dated item as platform-held is the safer direction to be wrong in — the operator is paid slightly later via Auto Payout rather than a pre-2027 item being silently destination-charged when it shouldn't be.

3.2 Tips are excluded from this round entirely

Every tip-collection call site — `StripeUtils::getStripeAttributesForAppTapToPay()` and `getStripeAttributesForCreateTip()` — passes `mode: PaymentModeResolver::DESTINATION` explicitly and unconditionally, regardless of tour date. This was tightened during this round specifically so it reads as a deliberate decision in the code, not an accident of both methods happening to predate the `$mode` parameter's default. Tips stay on today's destination-charge model until a separate, later round makes an explicit decision to change that.

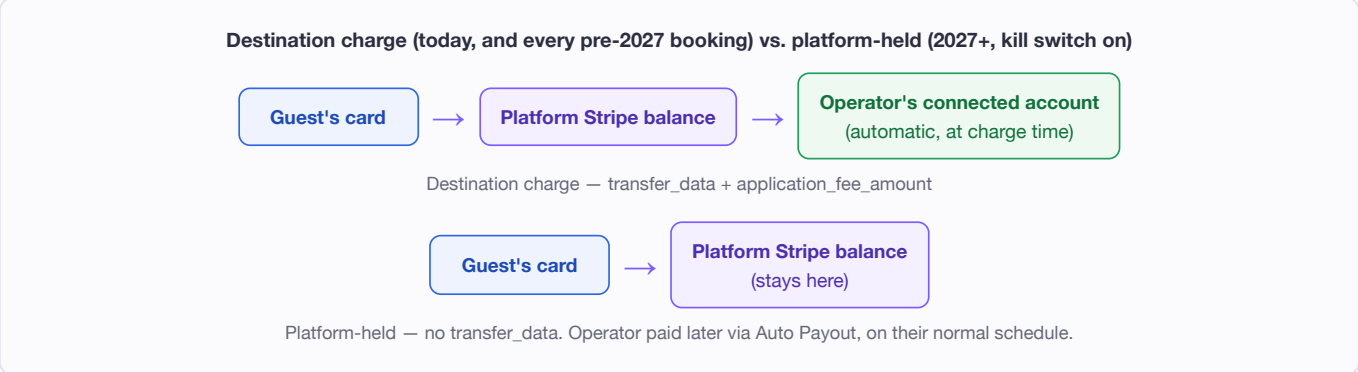
3.3 Auto Payout's own payout mechanism is not part of this round

This round makes sure a platform-held transaction is *excluded* from Auto Payout's existing sweep (Section 11) — it does not build the mechanism that will eventually pay the operator their share of a platform-held booking. That is intentionally deferred to a future round, once there is real production experience with platform-held transactions accumulating.

4. Target Architecture — Platform-Held Settlement

For a platform-held charge, `buildConnectedPaymentData()` omits `transfer_data` and `application_fee_amount` entirely — the full amount simply stays on the platform's Stripe balance, exactly as if the connected account didn't exist. The one thing that must survive unchanged is the guest-facing statement descriptor: the guest must see the tour operator's own name, never the platform's, on every booking regardless of settlement mode or country. Section 8 covers exactly how that requirement is met and the mistake made along the way; the resulting code is:

```
if ($mode === PaymentModeResolver::PLATFORM_HELD) {
    $data = [];
    if ($destinationId !== '' && $this->connectedAccountHasCardPayments($destinationId)) {
        $data['on_behalf_of'] = $destinationId;
        if ($statementDescriptor !== null) {
            $data['statement_descriptor_suffix'] = $statementDescriptor;
        }
    } elseif ($statementDescriptor !== null) {
        $data['statement_descriptor'] = $statementDescriptor;
    }
    return $data;
}
```



On refund, `buildRefundParams(array $params, string $mode)` strips `reverse_transfer` / `refund_application_fee` for platform-held mode — there is nothing on the original charge for either flag to act on, and sending them would error. It only ever removes these keys; it never adds either one, so a caller's own business logic (e.g. a refund type that deliberately keeps the platform's fee) is unaffected.

5. The 2027 Gate — `PaymentModeResolver` & Kill Switch

`App\Services\Payments\PaymentModeResolver` is the single place the 2027-01-01 boundary is expressed in code:

```
public const DESTINATION = 'destination';
public const PLATFORM_HELD = 'platform_held';
public const PLATFORM_HELD_STARTS_ON = '2027-01-01';
public const KILL_SWITCH_SETTING_KEY = 'PLATFORM_HELD_PAYOUT_ENABLED';

public function resolve(iterable $tourDates): string
{
    if (!hasSetting(self::KILL_SWITCH_SETTING_KEY)) {
        return self::DESTINATION;
    }
    foreach ($tourDates as $tourDate) {
        if ($this->normalizeDate($tourDate) >= self::PLATFORM_HELD_STARTS_ON) {
            return self::PLATFORM_HELD;
        }
    }
    return self::DESTINATION;
}
```

Two design choices are load-bearing and deliberate, not incidental:

- **Fails closed.** A tenant with no `PLATFORM_HELD_PAYOUT_ENABLED` row at all (an un-migrated or not-yet-opted-in tenant) gets `DESTINATION` unconditionally — the resolver does not even look at the dates. Rollout is therefore tenant-by-tenant and explicit, never automatic just because a calendar date passed.
- **Decided once, never re-derived.** Every call site resolves a mode at charge-creation time and writes it to `transactions.payment_mode`. Every later refund, cancellation, or dispute reads that column back rather than calling `PaymentModeResolver` again against the booking's current tour date — a booking's tour date can itself change (reschedule, switch package) after the original charge, so re-deriving later would silently disagree with what the original charge actually was. `PaymentModeInference::fromStripeObject()` exists for the few call sites that only have a raw Stripe object, not a `Transaction` row, in scope — it reads the object's own `transfer_data` / `transfer` presence as ground truth, which is a form of "read back what actually happened," not re-derivation from a date.

> **REAL NEAR-MISS WITH THE KILL SWITCH ITSELF**
>
> While writing this document, tenant `dixies` was found to already carry a `PLATFORM_HELD_PAYOUT_ENABLED` row with `value=1` — left over from an unrelated, already-reverted proof-of-concept on a prior branch that happened to reuse this exact settings key. Because the Foundation migration only inserts its safe default when the key doesn't already exist, this stale `enabled` row silently survived. No real charge was misrouted (zero `platform_held` transactions exist on any tenant, and no 2027 booking has been sold), but it is a concrete demonstration of why Section 9's rollout checklist requires confirming the kill switch's live value on a tenant before treating the migration's default as sufficient. A fix-up migration (`2026_09_18_000003_fix_stale_platform_held_payout_setting.php`) resets any row whose comment doesn't match this feature's own canonical text back to `0`.

6. New & Changed Components — Foundation

6.1 `PaymentModeResolver` / `PaymentModeInference` shipped

`app/Services/Payments/PaymentModeResolver.php` and `PaymentModeInference.php`. Covered in full in Section 5. 9 unit tests in `tests/Unit/Payments/PaymentModeResolverTest.php`.

6.2 `buildConnectedPaymentData()` / `buildRefundParams()` / `refundPayment()` shipped

`PaymentGatewayInterface` and `StripePaymentGateway`. Charge-creation side defaults `$mode` to `DESTINATION` (a forgotten mode reproduces today's exact behavior — a safe default). Refund side (`buildRefundParams(array $params, string $mode)`, `refundPayment(array $params, string $mode)`) makes `$mode` required with no default, so a call site that forgot to thread it through fails loudly (missing-argument error) instead of silently sending the wrong reversal flags. 8 tests in `tests/Unit/Payments/StripePaymentGatewayModeTest.php`.

6.3 `connectedAccountHasCardPayments()` shipped

Private method on `StripePaymentGateway`, a real Stripe API call (`$this->stripe->accounts->retrieve($accountId)`) checking `$account->capabilities->card_payments === 'active'`, statically cached per request. Used by both the destination-mode branch (pre-existing, unchanged) and the platform-held branch (new, Section 8) of `buildConnectedPaymentData()`.

6.4 `transactions.payment_mode` column & kill-switch setting shipped

Three tenant migrations: `2026_09_18_000001_add_platform_held_payout_setting.php` (kill switch, defaults off, `exists()`-guarded insert), `2026_09_18_000002_add_payment_mode_to_transactions_table.php` (enum column, default `'destination'`), and `2026_09_18_000003_fix_stale_platform_held_payout_setting.php` (the `dixies` stale-row correction described in Section 5).

6.5 `TransactionService` & `CancelTransactionUpdateService` mode threading shipped

Every method in `TransactionService` that writes a `transactions` row now threads `payment_mode` from the caller's already-resolved mode or already-in-scope `Transaction`. `CancelTransactionUpdateService::run()`'s `$mode` parameter is required (no default) across all 29 call sites project-wide, for the same "fail loud, not silent" reason as the refund-side gateway methods.

7. Phase-by-Phase Wiring (Card / Card Reader / Payment Link)

7.1 Phase A — Card (online) shipped

`app/Utils/StripeUtils.php`'s PaymentIntent-creation methods resolve mode from the tour date(s) already in scope at each site and thread it into `buildConnectedPaymentData()`. The two tip call sites (`getStripeAttributesForAppTapToPay`, `getStripeAttributesForCreateTip`) explicitly force `DESTINATION` per the Section 3.2 decision rather than inheriting the method's own default. `getStripeAttributesForAppCardReader()` takes an explicit `$mode` parameter from its caller. The full charge → refund → partial-refund → cancellation lifecycle for a platform-held booking was proven through this phase, since the refund/dispute infrastructure (Section 6) is channel-agnostic and only needed proving once.

7.2 Phase B — Card Reader shipped

Two structurally distinct flows, both fixed in the same pass:

- **Web/admin Stripe Terminal** (`CardReaderRepository::getPaymentIntent()`). A new private `resolveModeFromInput(array $input)` reads `metadata.dates` (cart context) or resolves a single tour date via `metadata.booking_id`, falling back to `DESTINATION` when neither is present:

  ```
  private function resolveModeFromInput(array $input): string
  {
      $dates = $input['metadata']['dates'] ?? null;
      if (is_array($dates) && !empty($dates)) {
          return app(PaymentModeResolver::class)->resolve($dates);
      }
      $bookingId = $input['metadata']['booking_id'] ?? null;
      if (!empty($bookingId)) {
          $tourDate = TourBooking::with('tourSlotTime')->find($bookingId)?->tourSlotTime?->date;
          return app(PaymentModeResolver::class)->resolveForSingleDate($tourDate);
      }
      return PaymentModeResolver::DESTINATION;
  }
  ```

  A pre-existing, independent bug was fixed in the same method: a hardcoded `'application_fee_amount' => $input['fees'] ?? 0` in the base payload, written *before* `buildConnectedPaymentData()`'s return value was spread in, meant this stray key would have survived into a platform-held charge even after mode support was added. Removed outright, not patched with a trailing `unset()`.
- **Mobile-app Terminal reader** (`TourBookingRepository::stripeCreatePaymentIntent()`, V1 and V3, both cart and single-booking branches) — tour date was already available at each site; wired directly.

JS payloads updated to supply the new field: `backend-card-reader.js` sends `meta.dates` derived from the cart's own package dates (or the single booking's date); `tour-booking-pos.js` sends the same from its own selected-packages list. 4 tests in `tests/Unit/Payments/CardReaderPlatformHeldModeTest.php`.

7.3 Phase C — Payment Link shipped

`PaymentLinkService::createPaymentLink($token)` rebuilds its Stripe Checkout Session fresh on every visit rather than once at link-creation time, so mode must be (and is) resolved inside the method itself at each rebuild, across the link's own booking dates, and threaded into both `buildConnectedPaymentData()` and `stripOnBehalfOfForCountry($params, $mode)` (Section 8). 2 tests in `tests/Unit/Payments/PaymentLinkPlatformHeldModeTest.php`.

> **NOT IN THIS PHASE**
> `CustomerController::createTipPaymentLink()` is a guide-tip link, not a booking payment link — it stays on `DESTINATION` unconditionally per Section 3.2, though it still calls the now-mode-aware `stripOnBehalfOfForCountry()` helper, passing `DESTINATION` explicitly.

8. The Canada Statement-Descriptor Investigation

The hard requirement behind this whole feature, stated by the business from the start, is that the guest must always see the tour operator's own name on their card statement — never the platform's — regardless of settlement mode or country. The mechanism for this is `on_behalf_of`: it marks the connected account as the entity the charge is "for," which is what determines whose name becomes the base of the statement descriptor. `statement_descriptor_suffix` only *appends* to whatever base was already selected by `on_behalf_of` — it cannot substitute for it.

8.1 The earlier, wrong conclusion

An earlier verification pass in this project tested whether a charge carrying `on_behalf_of` with no `transfer_data` at all — the exact shape platform-held mode needs — would succeed and correctly show the connected account's name, including against a Canadian test connected account (whose capabilities are `{"transfers": "active"}` only — no `card_payments`). Every test in that pass appeared to succeed. The conclusion drawn was that `on_behalf_of` works unconditionally with no `transfer_data`, regardless of `card_payments` capability, so `buildConnectedPaymentData()`'s platform-held branch could include `on_behalf_of` unconditionally whenever a destination ID exists, with no capability gate at all.

> **WHAT WAS ACTUALLY BEING TESTED**
>
> Every one of those earlier test PaymentIntents either never had a payment method confirmed against it, or reached `requires_action` for 3D Secure and stopped there. Stripe does not run its `on_behalf_of`/`card_payments` capability check until it actually attempts to create the underlying charge — an unconfirmed or 3DS-stalled PaymentIntent never reaches that point. The earlier pass had verified that Stripe would *accept the parameters at PaymentIntent-creation time*, not that a real charge would *complete* with them.

8.2 How the mistake was caught

The user provided a real Stripe Dashboard screenshot of a test transaction stuck in an `Incomplete` state with a blank "Statement descriptor" field, and asked for proper re-verification in the CA test environment rather than trusting the earlier result. Re-running the test with `off_session: true` and an explicit `confirm` call — forcing Stripe past 3D Secure to a real, completed (or genuinely rejected) charge — surfaced Stripe's actual error for the first time:

```
You cannot create a charge with the `on_behalf_of` parameter set to a
connected account with `transfers` but without the `card_payments`
capability enabled.
```

This meant the earlier conclusion was backwards for exactly the accounts most likely to need the fallback: a Canadian connected account with only `transfers` capability would have every platform-held charge rejected outright the moment a real card was charged, not merely display an imperfect descriptor.

8.3 The correct mechanism, verified

`statement_descriptor` (the full override, as opposed to the suffix) is a property of the **charging account** — the platform's own account — not the connected account. It requires no connected-account capability at all, because Stripe never needs to check what the connected account can do; the platform is the one making the charge. `buildConnectedPaymentData()`'s platform-held branch was rewritten to gate `on_behalf_of` on `connectedAccountHasCardPayments()` (the same check the destination-mode branch already used) and fall back to a full `statement_descriptor` override — set to the tour operator's own name — when the connected account lacks the capability:

```
if ($destinationId !== '' && $this->connectedAccountHasCardPayments($destinationId)) {
    $data['on_behalf_of'] = $destinationId;
    if ($statementDescriptor !== null) {
        $data['statement_descriptor_suffix'] = $statementDescriptor;
    }
} elseif ($statementDescriptor !== null) {
    $data['statement_descriptor'] = $statementDescriptor;
}
```

Verified end-to-end with real, completed test charges (not just PaymentIntent creation) against both `dixies` (US, has `card_payments` — takes the `on_behalf_of` path) and `aswta` (CA, does not — takes the `statement_descriptor` fallback path), confirming the connected account's own name appears on `Charge::calculated_statement_descriptor` in both cases, never the platform's. *Correction (Section 8.5): those charges called Stripe directly with the descriptor supplied; the booking code paths did not supply it until October 6, 2026.*

8.4 Why the pre-existing `stripOnBehalfOfForCountry()` helper is unrelated

`app/helpers.php`'s `stripOnBehalfOfForCountry(array $params, string $mode)` predates this project by about four months — traced via `git log --follow -p` to commit `7ba28a9`, "hotfix for Canada" (May 18, 2026), a real production fix for a completely different problem: Stripe rejecting cross-border `on_behalf_of` specifically when paired with `transfer_data` for a Canadian connected account. Its original test tenant, `mahikan`, no longer exists in the system; `aswta` is now the only Canadian tenant. It is called from exactly two places — `CustomerController.php`'s tip link and `PaymentLinkService::createPaymentLink()` — and only strips `on_behalf_of` when `$mode === PaymentModeResolver::DESTINATION`, which is exactly its original, unchanged behavior for every pre-2027 booking. It plays no role in the platform-held fix above; the two are separate mechanisms solving two different Stripe rejections that happen to both involve Canada.

> **NET EFFECT FOR CANADIAN OPERATORS UNDER PLATFORM-HELD MODE**
> A Canadian tour operator's guests see the operator's own name on their statement exactly as they do today — via `statement_descriptor` instead of `on_behalf_of` + suffix, a mechanical difference invisible to the guest. No business/requirements conversation was needed once the correct mechanism was identified; it was a straightforward code fix, confirmed working against a real account. *Correction (Section 8.5): this held for Payment Links only until October 6, 2026.*

8.5 Correction (October 6, 2026): the fix reached Payment Links only

Sections 8.3 and 8.4 describe the mechanism correctly but overstate where it was applied. The rewritten branch of `buildConnectedPaymentData()` produces a full `statement_descriptor` only when the caller passes the operator's descriptor to it as an argument. Of the 19 call sites, only `PaymentLinkService::createPaymentLink()` does. The other 18 — online card, POS, card reader (web and mobile app), and the V1/V3 booking repositories — set `statement_descriptor_suffix` on their own base payload and pass no descriptor to the helper, so for a platform-held charge on an account without `card_payments` the helper returned nothing for the descriptor. This was equally true at commit `422dc3ddf`.

> **WHY IT WAS MISSED**
> `StripePaymentGatewayModeTest` calls the helper directly and hands it a descriptor (`'MY TOUR CO'`), and the "end-to-end" charges in 8.3 were created by calling Stripe directly, so neither exercised the real booking code. Reproduced on October 6, 2026 in Stripe test mode with the real call-site shape on `aswta`, sent straight to Stripe: the guest statement was `TOURS* ASWTA` — the platform's own name (the platform's static descriptor is `TOURS`).

ROUTE (STRIPE TEST MODE)	BEFORE	NOW
Online card, Canadian operator (`aswta`, real call-site shape)	`TOURS* ASWTA`	`ASWTA`
Card reader / Terminal, simulated reader (test name `ASWTA TOURS`)	`TOURS* ASWTA TOURS`	`ASWTA TOURS`
Payment Link (Checkout)	already correct — descriptor stored on the PaymentIntent	unchanged
US operator with `card_payments` (`dixies`, `on_behalf_of`)	`DIXIESLOWE* DIXIES`	unchanged

The fix. It is applied once, at the single place every card PaymentIntent is created: `StripePaymentGateway::createPayment()` now calls the new `PlatformChargeStatementDescriptor::normalize()`. That rewrites only a charge made directly on the platform account — no `transfer_data`, no `on_behalf_of`, no `stripe_account` option — moving the operator's name from `statement_descriptor_suffix` into a full `statement_descriptor`, cleaned to Stripe's rules (no `< > \ ' " *`, 5 to 22 characters, at least one letter). Destination charges, `on_behalf_of` charges and direct charges on a connected account are returned untouched. If both a suffix and a full descriptor are sent, Stripe uses the suffix, so the suffix is removed, not supplemented. `PaymentLinkService` is unchanged.

LIMITS TO KNOW ABOUT

Docs vs. behaviour. Stripe's documentation says a full `statement_descriptor` is for non-card payments only; in test mode it is accepted for card charges, online and Terminal, and replaces the whole line. A sandbox test fails loudly if that changes; live mode is re-checked at go-live. **Short names.** A name under 5 characters cannot be a valid full descriptor, so the suffix is kept and the platform prefix would show; the only such account in the development database, `aswta`, is exactly 5 characters. **Checkout.** A completed hosted Checkout charge has still not been walked end-to-end, though Stripe stores the descriptor on the PaymentIntent.

9. Live Production Incidents Found Along the Way

None of the following are Payout Simplification bugs — all three predate this project — but all three were found only because this project required reading every refund/cancellation call site closely enough to notice something didn't reconcile. Recorded here because they were fixed in the same branch and window of work.

9.1 "Cannot reverse transfer on charge ... it does not have an associated transfer"

A cancellation attempt failed outright because a raw ledger-write call site had never recorded `payment_mode` at all, defaulting to `'destination'` for a charge that Stripe never actually gave a transfer to reverse. Root-caused to the same class of gap fixed comprehensively in Section 6.5 and 9.3 below — a raw `DB::table('transactions')` insert or a `TransactionService` call missing its mode.

9.2 "Refund amount ($180.49) is greater than unrefunded amount on charge ($102.92)" & the $94.47-vs-$86.02 follow-up

`TicketRefundCancelService.php`'s net-refund calculation added a "kept fee" bookkeeping figure (`$oldRefundKeepUs`, a pure reporting artifact from an unrelated pax-removal code path, never actually refunded to the guest) back into a real Stripe refund amount instead of only netting out the prior refund:

```
// before
$net_amount = $transTicketCost + $oldRefundKeepUs;
// after
$net_amount = $transTicketCost - $ref;
```

A structurally identical bug in `WeatherRefundCancelService.php` required netting out *both* the prior refund and a previously-kept fee (`$alreadyKeptFee`, summed from the booking's own `RefundHistory` rows) — an initial fix that only subtracted the prior refund still overpaid, caught by the user re-deriving the expected number by hand from the booking's real history before accepting the fix.

9.3 payment_mode silently defaulting to 'destination' on every cancellation refund

Covered in full in the Revision History and Section 6.5 — `CancelTransactionUpdateService::run()` never set `payment_mode` at all across any of its 29 call sites, so every cancellation refund row silently inherited the DB column's `'destination'` default. 8 already-wrong rows on `kens` were found by cross-referencing `status='Refund'` rows against sibling `status='Completed'` rows sharing the same `transaction_number`, and corrected directly in production.

10. The fee_amount / application_fee_id Correction

Scope, as clarified directly by the business owner mid-task after an initial broader framing was corrected: `fee_amount` / `application_fee_id` must be stored as `null` — never `0` or an empty string — whenever a refund type deliberately does **not** refund the platform's fee (Ticket Refund, Don't Refund, Partial Refund). A genuine 100%/Full Refund, which does refund the fee, must keep storing its real value, and was explicitly confirmed already correct and out of scope for this change.

Fixed at the shared-method level rather than auditing every call site individually, given the caller count (71 across the two `TransactionService` refund methods alone) — a deliberate trade-off favoring completeness over per-site precision:

```
// CancelTransactionUpdateService::run()
$feeAmountToStore = ((float) ($feeAmount ?? 0)) > 0 ? $feeAmount : null;
$applicationFeeIdToStore = ($feeAmountToStore !== null && $applicationFeeId !== '') ? $applicationFeeId : null;
```

The same pattern (a local computed value replacing the raw input before the write) was applied inside `TransactionService::storeRefundInTransaction()`, `storeAddonsRefundInTransaction()`, and `storeCustomChargeRefundInTransaction()`. 6 new tests in `TransactionServiceTest.php` and 3 in `CancelTransactionUpdateServiceFeeAmountTest.php` cover the null-input, zero-normalized-to-null, and genuine-non-zero-fee-preserved cases for each method.

11. Auto Payout Exclusion

Auto Payout pays operators via a real Stripe Connect payout drawn from their own connected-account balance. A destination charge auto-populates that balance via `transfer_data` the moment it succeeds; a platform-held charge never does — the money stays on the platform's own balance instead. `PayoutApprovalService::calculateAccountPayoutAmount()` and its siblings compute a payout amount by summing local `transactions` / `tour_bookings_master` rows, with no inherent awareness of which settlement model produced any given row.

Three query sites carry an explicit `payment_mode` filter for exactly this reason:

```
->where('tra.payment_mode', PaymentModeResolver::DESTINATION)
```

FILE	METHOD
`app/Services/PayoutApprovalService.php:109`	`calculateAccountPayoutAmount()` (or equivalent computation query)
`app/Services/PayoutAggregateService.php:189`	Mirrors `PayoutApprovalService`'s own filter, same reasoning
`app/Http/Controllers/Admin/AutoPayout/AutoPayoutController.php:637`	Admin-facing payout preview/execution query

WHY THE LIVE-BALANCE CHECK ALONE ISN'T SUFFICIENT

`PayoutApprovalService::executeImmediatePayout()`/`executeSinglePayout()` also check the connected account's **live** Stripe balance (`Balance::retrieve(['stripe_account' => $stripeAccountId])`) before sending a payout, as a blunt "don't overdraw" safety net. That check is unaware of *where* the balance came from — if the account happens to hold unrelated legitimate balance (a prior destination-charge booking, a manual top-up), it would not by itself catch a platform-held transaction being incorrectly included in the computed payout amount. The `payment_mode` filter above is the actual correctness fix; the balance check is a separate, coarser safety net that remains valuable but was never a substitute for this filter.

Building the mechanism that eventually *does* pay the operator their share of a platform-held booking (whatever that ends up looking like — likely a new Auto Payout leg once real production experience with platform-held transactions exists) is explicitly future work, not part of this round.

12. Data Model Changes

TABLE	CHANGE	MIGRATION
`transactions`	New `payment_mode` enum column (`'destination'` / `'platform_held'`), default `'destination'` , positioned after `via`	`2026_09_18_000002_add_payment_mode_to_transactions_table.php`
`settings` (tenant)	New `PLATFORM_HELD_PAYOUT_ENABLED` key/value row, default `0` , `exists()` -guarded insert	`2026_09_18_000001_add_platform_held_payout_setting.php`
`settings` (tenant, `dixies` only)	Corrects a stale `value=1` row left by an unrelated, already-reverted proof-of-concept that reused this same key	`2026_09_18_000003_fix_stale_platform_held_payout_setting.php`

13. Testing Summary

TEST FILE	COVERAGE	CASES
`tests/Unit/Payments/PaymentModeResolverTest.php`	Kill-switch fail-closed behavior, single-date and mixed-cart boundary resolution	9
`tests/Unit/Payments/StripePaymentGatewayModeTest.php`	`buildConnectedPaymentData()` for both modes, against real tenant Stripe destination IDs (`dixies` / `aswta`), including the capability-fallback case	8
`tests/Unit/Payments/CardReaderPlatformHeldModeTest.php`	`metadata.dates` / `metadata.booking_id` resolution and the no-metadata fallback	4
`tests/Unit/Payments/PaymentLinkPlatformHeldModeTest.php`	Per-rebuild mode resolution in `createPaymentLink()`	2
`tests/Unit/Bookings/TransactionServiceTest.php`	`fee_amount` / `application_fee_id` null-vs-real normalization across 3 refund methods (plus 9 pre-existing payment-write tests, unaffected)	6 new / 15 total
`tests/Unit/Bookings/CancelTransactionUpdateServiceFeeAmountTest.php`	Null input, zero-normalized-to-null, and genuine non-zero fee preservation for `run()`	3
`tests/Unit/Payments/PlatformChargeStatementDescriptorTest.php`	Which PaymentIntent shapes are rewritten to a full descriptor, and which (destination, `on_behalf_of` , direct charge, explicit descriptor, invalid name) are left alone — Section 8.5	12
`tests/Unit/Payments/PlatformHeldStatementDescriptorStripeTest.php`	Real Stripe test-mode charge: a bare suffix shows the platform name; through the gateway only the operator's. Skipped unless the key is `sk_test_` — Section 8.5	1

All new tests were verified with the standard revert-confirm-restore discipline used throughout this branch: temporarily revert the fix via `git stash push -- <file>`, confirm the new/updated test fails with the exact expected error, restore the fix, confirm it passes again.

14. Risk Register

RISK	MITIGATION	STATUS
A tenant's kill switch reads as enabled when it shouldn't, routing a real charge to platform-held mode prematurely	Fails closed by default; the one real instance found (`dixies`) was a stale row from an unrelated feature, now corrected by migration; rollout checklist requires confirming live value per tenant before relying on the default	**mitigated, verify per tenant before enabling**
A call site threads a fresh `PaymentModeResolver` result at refund/cancellation time instead of reading the persisted `payment_mode`	Architectural principle stated repeatedly in this document; `PaymentModeInference` exists specifically to read ground truth off a raw Stripe object rather than re-deriving from a possibly-changed tour date	**designed against, spot-checked via the 29-site and 71-caller sweeps**
A Canadian (or any non-US) connected account's guests see the platform's name instead of the operator's under platform-held mode	Verified end-to-end with a real completed charge against a live CA test account; the earlier false-positive and its correction are documented in full in Section 8 specifically so this class of mistake is checked for, not just this instance of it. *Coverage corrected October 6, 2026 (Section 8.5): the fix first reached Payment Links only; it now applies at the gateway to every route.*	**verified against a real charge**
Auto Payout pays an operator for money that never left the platform's balance	`payment_mode = 'destination'` filter on all 3 relevant query sites (Section 11)	**shipped**
A full Checkout-Session hosted-page redirect flow (as opposed to a direct API-confirmed PaymentIntent) has not been exercised end-to-end for the platform-held + CA-fallback combination	None yet — this environment cannot drive a real hosted Stripe checkout page interactively	**open, see Section 15**

15. Open Items — What's Pending

- **Statement-descriptor behaviour in live mode (added October 6, 2026).** The full-descriptor approach contradicts Stripe's documentation (card payments are documented as suffix-only) although it works in test mode for online and Terminal charges; a sandbox test guards it. Confirm with the first small real charge at go-live (Section 8.5).
- **Short operator descriptors (added October 6, 2026).** A statement descriptor under 5 characters cannot be sent as a full line, so a Canadian-style operator with one would still show the platform prefix. Check any new such operator before enabling the kill switch.
- **Full Checkout-Session-redirect verification.** All statement-descriptor verification so far used direct API PaymentIntent confirmation (with `off_session: true` to force past 3DS). Payment Link's flow is a real hosted Checkout Session redirect, which has not been separately walked end-to-end as a completed charge — worth doing before enabling the kill switch on a tenant that relies heavily on payment links.
- **Auto Payout's "pay the operator for a platform-held booking" leg.** Explicitly out of scope for this round (Section 3.3, Section 11) — deferred until there is real production experience with platform-held transactions to design against.
- **Legacy Phase A2 raw-card-number paths** (older, possibly low-traffic `TourBookingRepository` creation methods that bypass `StripeUtils.php`) have not been individually re-verified against the newer `payment_mode`/fee-normalization fixes — worth a traffic check before assuming they need the same treatment.
- **Per-tenant kill-switch rollout.** No tenant currently has `PLATFORM_HELD_PAYOUT_ENABLED` intentionally turned on. Before enabling it anywhere: confirm the live setting value directly (per the Section 5 near-miss), and confirm that tenant's connected account's `card_payments` capability status so the expected statement-descriptor path (suffix vs. full override) is known in advance rather than discovered on the first real charge.

16. Appendix A — Key File & Call-Site Reference

COMPONENT	FILE
Mode resolver	`app/Services/Payments/PaymentModeResolver.php`
Ground-truth inference from a raw Stripe object	`app/Services/Payments/PaymentModeInference.php`
Connected-payment param builder, refund param builder, capability check	`app/Services/Payments/StripePaymentGateway.php`
Interface	`app/Contracts/Payments/PaymentGatewayInterface.php`
Canada on_behalf_of strip helper (pre-existing, unrelated origin)	`app/helpers.php` — `stripOnBehalfOfForCountry()`
Phase A — online card creation	`app/Utils/StripeUtils.php`
Phase B — web/admin Terminal	`app/Repositories/Pos/CardReaderRepository.php`
Phase B — mobile-app Terminal	`app/Repositories/TourBooking/TourBookingRepository.php` & `V3/TourBookingRepository.php` — `stripeCreatePaymentIntent()`
Phase B — JS payloads	`public/js/user-common/backend-card-reader.js`, `public/js/user-common/tour-booking-pos.js`
Statement-descriptor safety net (all routes, Section 8.5)	`app/Services/Payments/PlatformChargeStatementDescriptor.php`, called from `StripePaymentGateway::createPayment()`
Phase C — Payment Link	`app/Services/Booking/PaymentLinkService.php` — `createPaymentLink()`
Ledger writes (payment side)	`app/Services/Booking/TransactionService.php`
Ledger writes (cancellation-refund side, 29 call sites)	`app/Services/CancelBooking/CancelTransactionUpdateService.php`, `TicketRefundCancelService.php`, `WeatherRefundCancelService.php`, `PartialRefundCancelService.php`, `DontRefundCancelService.php`
Auto Payout exclusion	`app/Services/PayoutApprovalService.php`, `PayoutAggregateService.php`, `app/Http/Controllers/Admin/AutoPayout/AutoPayoutController.php`
Migrations	`database/migrations/tenant/2026_09_18_000001..3_*.php`

17. Appendix B — Stripe Connect Glossary

TERM	MEANING
Destination charge	A PaymentIntent created on the platform account with `transfer_data.destination` set; Stripe automatically splits and transfers funds to the connected account the moment the charge settles. Still the model for every pre-2027 booking, and every booking on a tenant with the kill switch off.
Platform-held charge	This project's term (not a formal Stripe product name) for a plain platform charge with no `transfer_data` / `application_fee_amount` at all — the full amount stays on the platform's own balance until moved later by a separate mechanism (here, Auto Payout, on its normal schedule).
on_behalf_of	Marks the connected account as the entity the charge is "for" — determines the base of the guest-facing statement descriptor and shifts some card-present liability. Requires the connected account to have `card_payments` capability active, **even with no `transfer_data` present** — the mistake corrected in Section 8.
statement_descriptor_suffix	Appends to whatever descriptor base `on_behalf_of` already selected. Cannot substitute for `on_behalf_of` — without it, there is no operator-name base for the suffix to append to.
statement_descriptor (full override)	A property of the **charging** (platform) account, not the connected account — replaces the descriptor entirely, independent of `on_behalf_of` and requiring no connected-account capability at all. The fallback mechanism for accounts without `card_payments` (Section 8.3).
card_payments capability	A connected account capability, separate from `transfers`. Most Canadian operators on this platform have only `transfers` active, not `card_payments` — the reason the Section 8 fallback exists at all.
application_fee_amount	On a destination charge, the slice the platform keeps; the rest transfers automatically. Omitted entirely for platform-held charges — meaningless once there's no automatic split to take a slice of.
reverse_transfer / refund_application_fee	Refund-time flags meaningful only against a destination charge. Stripped by `buildRefundParams()` for platform-held mode — sending either against a charge that never had `transfer_data` would error.